Universidad Nacional Autónoma de México
Facultad de Ingeniería
Arquitectura Cliente Servidor
Ortega de la Paz Rafael | 420054085
12 de Octubre del 2026

Tarea 4: listen(), backlog y colas de conexiones TCP

Introducción

En el presente trabajo se documenta la ejecución práctica del laboratorio propuesto en la Tarea 4 de la asignatura, relacionado con el encolamiento de conexiones TCP en un servidor de sockets AF_INET. Se utilizó el programa ejemplo_listen.c, que invoca a listen() con un backlog igual a 1 y atiende a un solo cliente a la vez; con el comando nc se lanzaron cuatro clientes mientras los sockets se monitoreaban con ss. Además, se presenta la investigación previa sobre sk_acceptq_is_full(), los parámetros tcp_syncookies y tcp_syn_retries, y las colas SYN y de aceptación (Román Zamitiz, s. f.).

Investigación previa

¿Qué hace la función sk_acceptq_is_full()? Es una función en línea de include/net/sock.h, en el kernel de Linux, que indica si la cola de aceptación de un socket en escucha está llena. Compara el número de conexiones ya establecidas que esperan a ser tomadas por accept() (sk_ack_backlog) con el máximo configurado (sk_max_ack_backlog), que proviene del argumento backlog de listen() (Linux man-pages project, 2026a; Torvalds et al., s. f.). La comparación es estrictamente "mayor que" y no "mayor o igual que": el comentario que acompaña al código advierte que cambiarla a "mayor o igual" fue una modificación incorrecta que tuvo que revertirse (Torvalds et al., s. f.). Por ello, con un backlog igual a n la cola acepta n + 1 conexiones antes de considerarse llena. Con n = 1, el cliente 1 es tomado de inmediato por accept(), de modo que al llegar el cliente 2 la cola tiene 0 conexiones (0 > 1 es falso) y entra; al llegar el cliente 3 tiene 1 (1 > 1 es falso) y también entra; y al llegar el cliente 4 tiene 2 (2 > 1 es verdadero) y se descarta (Román Zamitiz, s. f.). Cuando devuelve verdadero, el kernel descarta la conexión y registra el desbordamiento en el contador LINUX_MIB_LISTENOVERFLOWS (Torvalds et al., s. f.).

¿Qué hace el parámetro tcp_syncookies? Según tcp(7), habilita las cookies SYN, mecanismo que protege un socket contra ataques de inundación SYN (SYN flood); su valor predeterminado es 1. Con 0 están deshabilitadas, con 1 se envían cuando la cola de solicitudes SYN de un socket se desborda y con 2 (desde Linux 3.12) se envían siempre (Linux man-pages project, 2026b). En el laboratorio se desactivó, siguiendo la indicación del profesor (Román Zamitiz, s. f.).

¿Qué hace el parámetro tcp_syn_retries? Define el número máximo de retransmisiones del SYN inicial de un intento de conexión activo; por defecto es 6, equivalente a unos 127 segundos de reintentos (Linux man-pages project, 2026b). Como los clientes nc corrieron en la misma máquina, con el valor 1 retransmiten el SYN una sola vez y fallan en pocos segundos, en lugar de esperar más de dos minutos (Román Zamitiz, s. f.).

¿Qué son la cola SYN y la cola de aceptación? Para cada socket en escucha, el kernel mantiene dos colas durante el saludo de tres vías (Stevens et al., 2004, sección 4.5). Un SYN entrante pasa a la cola SYN, de conexiones incompletas (SYN_RECV), y se responde con SYN+ACK; cuando el cliente devuelve el ACK, la conexión pasa a la cola de aceptación, de conexiones establecidas (ESTABLISHED), donde espera a que la aplicación llame a accept() (OpenAnolis, 2022). Desde Linux 2.2, el argumento backlog de listen() fija el tamaño de la cola de aceptación, mientras que la cola SYN se controla con tcp_max_syn_backlog (Linux man-pages project, 2026a).

El desbordamiento de cada cola tiene causas y efectos distintos. La cola SYN se desborda cuando llegan más solicitudes de las que pueden mantenerse sin confirmar, como en un ataque SYN flood, y el kernel comienza a descartarlas (Linux man-pages project, 2026b). La cola de aceptación se desborda cuando la aplicación no llama a accept() con suficiente rapidez; entonces, según listen(2), el cliente puede recibir ECONNREFUSED o, en TCP, la solicitud puede ignorarse para que un reintento posterior tenga éxito (Linux man-pages project, 2026a). En el laboratorio, ejemplo_listen.c es un servidor secuencial que no regresa a accept() mientras atiende al cliente 1.

Actividad 1 — Configuración del entorno e inicio del servidor

Se abrieron seis terminales en el sistema, acomodadas en mosaico: la terminal 1 para el servidor, la terminal 2 para el monitoreo y las terminales 3 a 6 para los cuatro clientes. En la terminal 1 se consultaron los valores originales de los parámetros con cat /proc/sys/net/ipv4/tcp_syncookies y cat /proc/sys/net/ipv4/tcp_syn_retries, obteniendo 1 y 6, respectivamente. Después se modificaron con sudo sysctl -w net.ipv4.tcp_syncookies=0 y sudo sysctl -w net.ipv4.tcp_syn_retries=1, y se verificó el cambio repitiendo los cat, que devolvieron 0 y 1. Finalmente se ejecutó el servidor, previamente compilado con gcc ejemplo_listen.c -o ejemplo_listen, mediante ./ejemplo_listen 4898; el programa llama a listen(server_fd, 1) y queda esperando en accept().



Imagen 1: Terminal 1 — consulta y modificación de tcp_syncookies y tcp_syn_retries, e inicio del servidor ejemplo_listen en el puerto 4898

En la terminal 2 se ejecutó watch -n 1 "ss -atn 'sport = :4898'", que actualiza cada segundo los sockets cuyo puerto local es 4898. El socket aparece en estado LISTEN con Recv-Q igual a 0 y Send-Q igual a 1: para un socket en escucha, Recv-Q es el número de conexiones que esperan en la cola de aceptación y Send-Q es su longitud máxima, es decir, el backlog indicado en listen() (OpenAnolis, 2022).

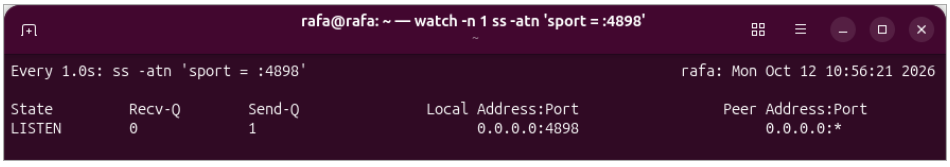


Imagen 2: Terminal 2 — monitoreo con watch y ss: socket en LISTEN con Recv-Q 0 y Send-Q 1

Actividad 2 — Cliente 1: conexión aceptada

Con el servidor y el monitoreo en marcha, en la terminal 3 se ejecutó nc -v 127.0.0.1 4898 (cliente 1). Netcat informó que la conexión se realizó con éxito (succeeded), lo que indica que el saludo de tres vías se completó.

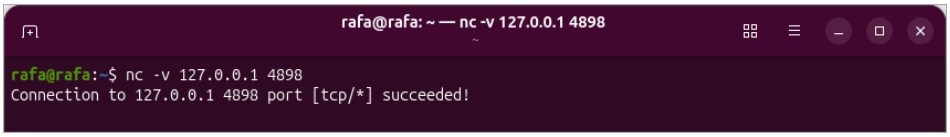


Imagen 3: Terminal 3 — cliente 1: conexión exitosa con nc

En la terminal 1, el servidor imprimió el mensaje "Cliente aceptado!": accept() devolvió la conexión y el servidor quedó ocupado atendiéndola dentro de su ciclo recv().

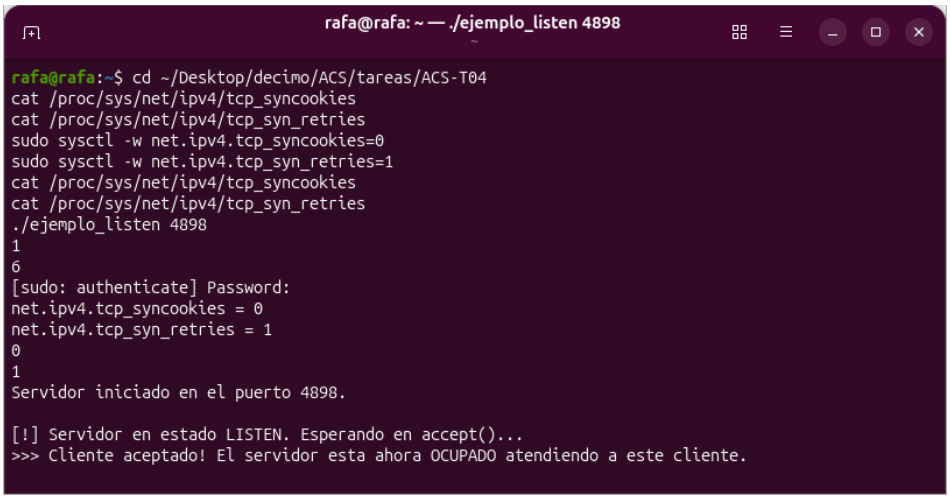


Imagen 4: Terminal 1 — el servidor acepta al cliente 1 y queda ocupado atendiéndolo

En la terminal 2 apareció una conexión ESTAB junto al socket en LISTEN, que conserva Recv-Q 0 y Send-Q 1: el cliente 1 ya salió de la cola de aceptación y la cola está vacía.

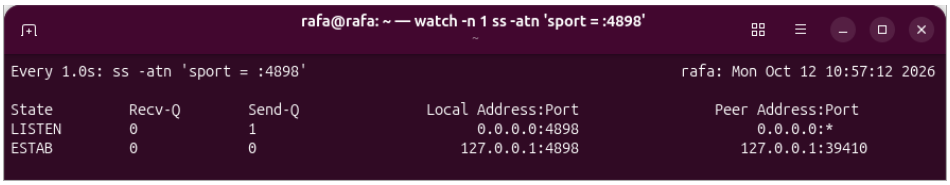


Imagen 5: Terminal 2 — una conexión ESTAB y Recv-Q 0, con el cliente 1 ya atendido

Actividad 3 — Clientes 2 y 3: conexiones en cola

El cliente 2 (terminal 4) también recibió succeeded, pero el servidor no imprimió nada nuevo porque sigue atendiendo al cliente 1.

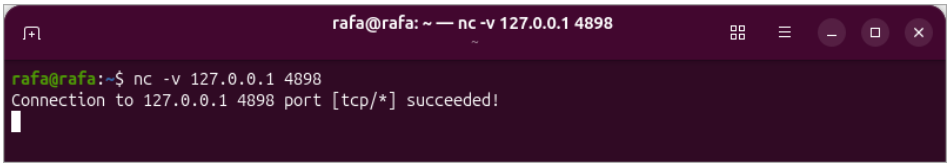


Imagen 6: Terminal 4 — cliente 2: conexión exitosa con nc

En la terminal 2, el Recv-Q del socket en LISTEN pasó a 1 y aparecen dos conexiones ESTAB: una atendida y otra esperando en la cola de aceptación.

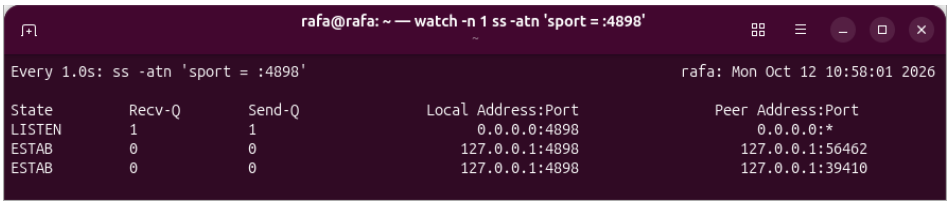


Imagen 7: Terminal 2 — Recv-Q 1 y dos conexiones ESTAB: el cliente 2 espera en la cola

Con un backlog de 1 podría esperarse que el cliente 3 (terminal 5) fuera rechazado, pero también recibió succeeded: la cola tenía 1 conexión y 1 > 1 es falso, por lo que sk_acceptq_is_full() aún no la consideró llena.

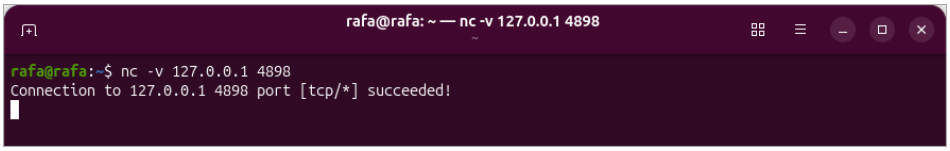


Imagen 8: Terminal 5 — cliente 3: conexión exitosa con nc

Ahora Recv-Q vale 2 y hay tres conexiones ESTAB: la atendida y las dos encoladas. Que Recv-Q (2) sea mayor que Send-Q (1) es la evidencia directa de la regla n + 1: la cola de aceptación contiene una conexión más que el backlog solicitado.

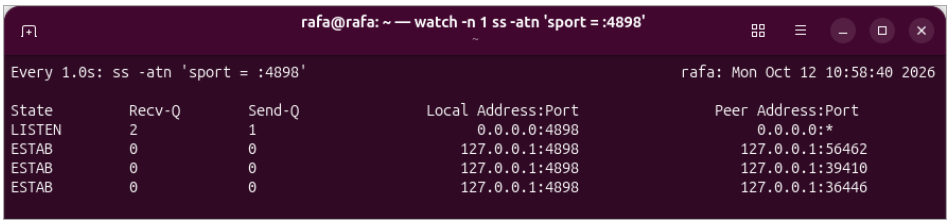


Imagen 9: Terminal 2 — Recv-Q 2 (mayor que Send-Q 1) y tres conexiones ESTAB

Actividad 4 — Cliente 4: conexión descartada

El cliente 4 (terminal 6) no logró conectarse: tras unos segundos, nc terminó con "Connection timed out". Con Recv-Q igual a 2, la comparación 2 > 1 es verdadera, la cola se considera llena y el kernel descarta la solicitud sin completar el saludo; como tcp_syn_retries vale 1, el cliente retransmite el SYN una sola vez y se rinde pronto.



Imagen 10: Terminal 6 — cliente 4: Connection timed out

En la terminal 2 no apareció ninguna conexión nueva: se mantienen Recv-Q 2 y las mismas tres conexiones ESTAB, pues el cliente 4 nunca estableció la conexión.

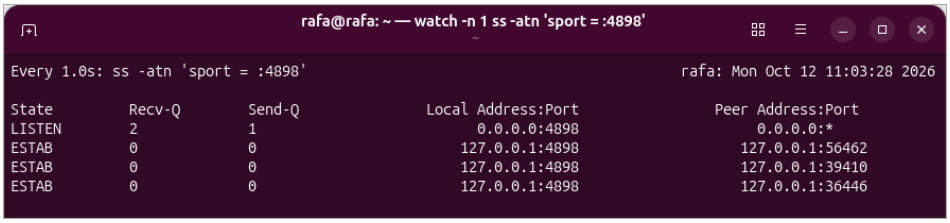


Imagen 11: Terminal 2 — tras el timeout del cliente 4, Recv-Q 2 y las mismas tres conexiones ESTAB

La siguiente captura reúne las seis terminales, como se solicita en el enunciado: servidor (1), monitoreo (2), clientes 1 a 3 con succeeded (3 a 5) y cliente 4 con el timeout (6).

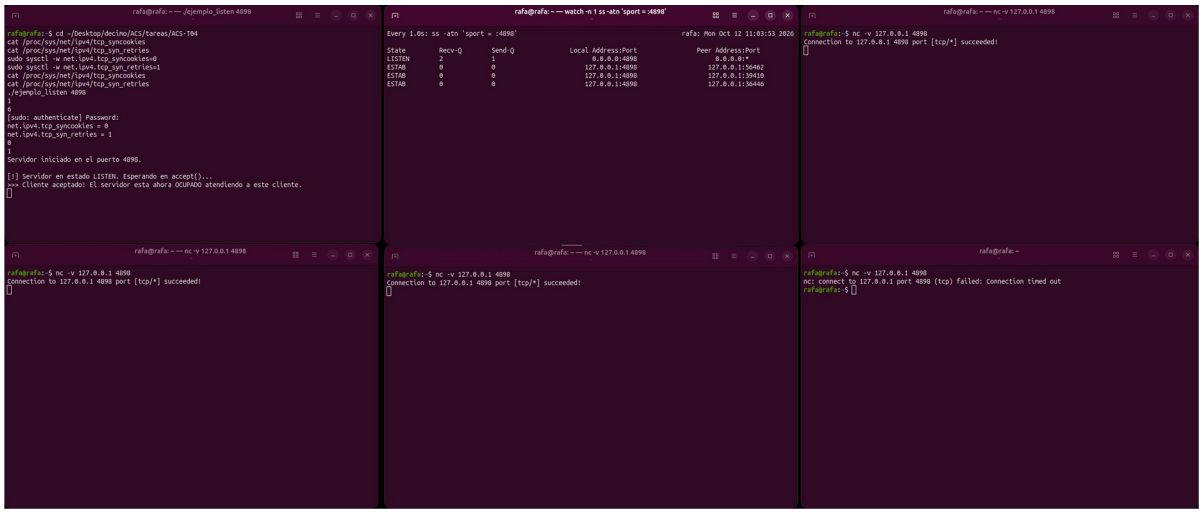


Imagen 12: Las seis terminales a la vez — servidor, monitoreo, clientes 1 a 3 con succeeded y cliente 4 con Connection timed out

Conclusión

A través de este laboratorio se comprobó que, con listen(server_fd, 1), el servidor atendió a un cliente y retuvo a otros dos en la cola de aceptación, mientras que el cuarto no logró establecer la conexión: se observaron tres conexiones ESTAB y un Recv-Q de 2 frente a un Send-Q de 1. Esto se explica porque sk_acceptq_is_full() compara con "mayor que" y no con "mayor o igual que", de modo que la cola admite backlog + 1 conexiones; por tanto, el backlog no equivale al número de clientes que un servidor puede atender ni a un límite exacto de espera. También se verificó que, con tcp_syncookies en 0 y tcp_syn_retries en 1, el descarte se manifiesta como un timeout de pocos segundos, y se distinguió entre la cola SYN, que retiene conexiones aún no confirmadas, y la cola de aceptación, que retiene conexiones ya establecidas que esperan a accept(). Finalmente, el ejercicio evidencia la limitación de un servidor secuencial, que no regresa a accept() mientras atiende a un cliente, y anticipa la necesidad de servidores concurrentes.

Referencias

Linux man-pages project. (2026a). *listen(2) — Linux manual page* (Versión 6.19) [Página de manual]. man7.org. https://man7.org/linux/man-pages/man2/listen.2.html

Linux man-pages project. (2026b). *tcp(7) — Linux manual page* (Versión 6.19) [Página de manual]. man7.org. https://man7.org/linux/man-pages/man7/tcp.7.html

OpenAnolis. (2022, 3 de agosto). *TCP SYN queue and accept queue overflow explained*. Alibaba Cloud Community. https://www.alibabacloud.com/blog/tcp-syn-queue-and-accept-queue-overflow-explained_599203

Román Zamitiz, C. A. (s. f.). *Tarea 4* [Documento PDF]. Arquitectura Cliente Servidor. https://cromanzamitiz.github.io/arquitectura-cliente-servidor/tema3/Tarea_4.pdf

Stevens, W. R., Fenner, B., & Rudoff, A. M. (2004). *UNIX network programming: Vol. 1. The sockets networking API* (3.ª ed.). Addison-Wesley.

Torvalds, L., et al. (s. f.). *Linux kernel source tree* [Código fuente: include/net/sock.h y net/ipv4/tcp_ipv4.c]. GitHub. Recuperado el 12 de octubre de 2026, de https://github.com/torvalds/linux